Ders 8 Çalışma Özeti

Ders 8 Çalışma Özeti

Genel Konular

- Bu derste fonksiyonlara giriş yapılır: neden gerektikleri, tanımlanmaları, çağrılmaları, prototipleri, değerle ve referansla çağırma. Ardından dizilerin fonksiyonlara aktarılması anlatılır. Son olarak bubble sort, sıralı arama ve ikili arama (binary search) fonksiyonlarla kodlanır. Dersin sonunda laboratuvar sınavının nasıl yapılacağı açıklanır.
- Neden fonksiyon?
  - Bir diziyi ekrana yazdıran kod, dizide her değişiklikten sonra tekrar kopyalanıyorsa bu bir "kod kokusu"dur (code smell). Kopyalanan kodda bir değişiklik gerekirse her kopyada ayrı ayrı yapılmalıdır. Kod büyüdükçe bu yönetilemez hâle gelir.
  - Çözüm, programı parçalara (modüllere) bölmektir. Modülerlik, yapısal programlamanın en büyük faydalarından biridir. `#define MAX 10` ile boyutu tek bir yerden değiştirmek gibi, bir işi tek bir fonksiyonda yazınca değişiklik de tek yerde yapılır.
  - C'de modüllere fonksiyon denir. Kütüphane fonksiyonları (`printf`, `scanf`, `sqrt`, `floor`, `ceil` vb.) ve kullanıcı tanımlı fonksiyonlar (user-defined functions) vardır.
  - Böl ve fethet (divide and conquer): büyük bir problem çözülemiyorsa küçük parçalara bölünür, parçalar çözülerek bütün çözülür.
  - Fonksiyonlar tekrar tekrar kullanılabilir. `printf` her seferinde yeniden derlenmez, derlenmiş hâli bağlanır.
- Fonksiyon çağrısı (function call, invoke)
  - Fonksiyonun adı yazılır ve parantez içinde argümanlar verilir. `printf` en az bir, `scanf` en az iki argüman alır.
  - "Invoke" bilgisayar bilimlerinde çağırmak demektir. "Revoke" iptal etmek, "evoke" tetiklemek (ör. kesmelerde) anlamında kullanılır.
  - Patron-işçi benzetmesi: patron işi verir, işçi yapar ve sonucu getirir. Patron işin nasıl yapıldığını bilmek zorunda değildir (information hiding). `printf`'in içinde ne olduğunu bilmeden kullanırız.
  - Çağrıda gönderilen değerlere argüman, fonksiyon tanımında bu değerleri alan değişkenlere parametre denir.
  - Bir C programında `main` başka fonksiyonları (worker) çağırır. Onlar da başka fonksiyonları çağırabilir. Veriler argüman olarak gider, sonuç dönüş değeri (return) olarak gelir veya gelmez.
- Fonksiyon tanımı (definition)
  - Biçim: `dönüş_tipi fonksiyon_adı(parametre listesi) { tanımlamalar ve ifadeler }`.
  - Parametre alıp almaması ve değer döndürüp döndürmemesine göre $2 \times 2 = 4$ durum vardır.
  - Değer döndürmeyen fonksiyonun dönüş tipi `void`'dir ("boş"). Ör. `void hello() { ... }`, çağrısı `hello();`.
  - Değer döndüren fonksiyon `return` ile tek bir değer döndürür. Ör. `int get() { ... return x; }`, çağrısı `t = get();`.
  - C'de bir fonksiyon yalnızca tek bir değer döndürebilir. `return a, b;` gibi birden fazla değer döndürülemez. Bir dizi de döndürülemez, ama dizinin bir elemanı döndürülebilir. `return x + y;` gibi bir ifadenin sonucu da döndürülebilir.
  - Parametreli fonksiyon: `int cube(int n) { return n * n * n; }`, çağrısı `t = cube(3);` veya doğrudan `printf("%d", cube(7));`. Dönen değer bir değişkene atanmak zorunda değildir.

  - Her parametrenin tipi ayrı ayrı yazılır: `int add(int a, int b)`. `int add(int a, b)` yazılamaz.
  - Dönüş tipi `void` olan bir fonksiyonda değer vermeden `return;` yazılabilir. Bu, fonksiyondan erken çıkmak için kullanılır.
  - Fonksiyon adları da değişken adlandırma kurallarına uyar. Ama bir eylemi ifade ettikleri için fiil gibi adlandırılır (`sirala`, `yazdir`).
  - `math.h`'deki matematik fonksiyonlarının dönüş tipi `double`'dır. Argüman sabit, değişken veya ifade olabilir: `sqrt(900)`, `sqrt(x)`, `sqrt(x + 2)`.
- Program akışı
  - Bir fonksiyon çağrısına gelince işlemcideki program counter fonksiyonun ilk satırına atlar. Geri dönülecek satır bir yerde saklanır. Fonksiyonun kapanış süslü parantezine gelince çağrının yapıldığı yere dönülür ve sonraki satırdan devam edilir.
- Yerel değişkenler (local variables)
  - Fonksiyonun parametreleri ve içinde tanımlanan değişkenler o fonksiyona özeldir. Başka bir fonksiyondan erişilemez. Fonksiyon bitince serbest kalırlar.
  - `main`'deki `x` ile başka bir fonksiyondaki `x` farklı değişkenlerdir. Aynı adı kullanmak sorun değildir.
  - C'de değişkenler, işlemlerden önce, bloğun başında tanımlanır.
- Düşük bağlaşım, yüksek uyum (low coupling, high cohesion)
  - Bir fonksiyon tek bir iş yapmalıdır. Örneğin diziyi sıralayan fonksiyon yazdırmamalıdır. 3 milyon elemanı sıralayıp yazdırmak ağır bir yüktür. Yazdırmak isteyen ayrıca `yazdir` fonksiyonunu çağırır.
  - Fonksiyonun adı ile yaptığı iş uyumlu olmalıdır. `sirala` adlı bir fonksiyonda matris çarpımı yapılmamalıdır.
  - Düşük bağlaşım ve yüksek uyum anlaşılmayı, yeniden kullanılabilirliği ve bakımı kolaylaştırır. Bir değişiklik diğer parçaları daha az etkiler.
- Fonksiyon prototipi (function declaration)
  - Fonksiyonun tanımı `main`'den sonra yazılıyorsa, `main`'den önce derleyiciye fonksiyonun adı, parametre tipleri ve dönüş tipi bildirilir: `int square(int);`.
  - En kolay yol: fonksiyonun başlık satırını kopyalayıp yukarıya yapıştırmak, sonuna noktalı virgül koymak ve parametre adlarını silmek. Parametre adları yazılsa da çalışır, ama tipleri yazmak yeterlidir.
  - Prototip, fonksiyonun çağrısından da çıkarılabilir: gönderilen argümanların ve dönüş değerinin yazdırıldığı biçim belirtecinin (`%d`) tiplerine bakılır.
  - Fonksiyonun tanımı `main`'den önce yazılırsa prototipe gerek kalmaz. Prototip yazılmazsa bazı derleyiciler hata, bazıları uyarı verir. İkinci sınıfta prototipler başlık (header) dosyalarına taşınacaktır.
  - Argüman tipi parametre tipinden farklıysa gizli dönüşüm olur. Ör. `int` bekleyen fonksiyona 3.6 gönderilirse değer `int`'e çevrilir ve kayıp oluşur. Bu genellikle derleme hatası vermez, sonuç hatası olarak ortaya çıkar.
  - `printf` ve `scanf`'teki biçim belirteçleri (`%d`, `%lf`, `%c`) de bu fonksiyonların prototiplerine ve iç kontrollerine uygun kullanım içindir.
- Değerle çağırma (call by value)
  - Argümanın değeri parametreye kopyalanır. Bellekte iki ayrı kopya olur. Fonksiyonun içinde parametrede yapılan değişiklik orijinal değişkeni etkilemez. Bu, kazara yapılan değişikliklerin önüne geçer.
  - `printf`'e gönderilen değer değiştirilmez, yalnızca yazdırılır. Bu bir değerle çağırmadır.
- Referansla çağırma (call by reference)
  - Değişkenin adresi gönderilir. Fonksiyon orijinal değişkeni değiştirebilir. `scanf("%d", &n)` böyle çalışır: okunan değer `n`'in bellekteki adresine yazılır.
  - `&` unutulursa (`scanf("%d", n)`), `n`'in değeri bir adres gibi kullanılır. Örneğin 14 girilmişse 14 numaralı adrese yazılmaya çalışılır ve işletim sistemi programı hatayla durdurur.
  - Referansla çağırma, bir fonksiyondan birden fazla sonuç almanın da yoludur. `scanf("%d %d", &a, &b)` iki değişkeni birden değiştirir.
  - C'de gerçek bir referansla çağırma yoktur. Bu, adres (pointer) ile yapılır. Hoca buna "yalancı call by reference" der. Ayrıntılar pointer konusunda görülecektir.
- Diziler
  - Diziler bellekte ardışık saklanır. İlk eleman 0. pozisyonda, son eleman `n-1`. pozisyon-

dadır.
  - Dizinin adı, 0. elemanının adresini verir. `dizi`, `&dizi[0]` ve `&dizi` aynı adresi yazdırır.
  - İlk değer verme: `int n[5] = {1, 2, 3, 4, 5};`. Daha az eleman verilirse kalanlar 0 olur. `int n[5] = {0};` bütün elemanları 0 yapar. Boyut yazılmazsa (`int n[] = {1, 2, 3};`) verilen eleman sayısı kadar yer ayrılır.
  - Karakter dizileri (string): `char s[] = "first";` sonuna otomatik olarak sonlandırma karakteri `'\0'` eklenir, bu yüzden 6 karakterlik yer kaplar. `char s[] = {'f', 'i', 'r', 's', 't', '\0'};` ile eşdeğerdir. Karakter karakter ilk değer verilirken `'\0'` elle eklenmezse, yazdırma bellekteki anlamsız karakterlerle devam edebilir.
  - `scanf("%s", s)`'de & gerekmez, çünkü dizinin adı zaten adrestir. `%s` boşluk görünce durur: "Hello there" girilince yalnızca "Hello" okunur.
  - Bir string, `'\0'` görülene kadar `%c` ile karakter karakter de yazdırılabilir.
- Dizilerin fonksiyonlara aktarılması
  - Dizi, adı ve boyutu ile gönderilir: `modifyArray(myArray, 24);`.
  - Parametre tarafında boyut yazılmaz: `void modifyArray(int b[], int size)`. Prototipte `void modifyArray(int [], int);`.
  - Dizi, ilk elemanının adresi gönderildiği için referansla aktarılır. Fonksiyonda dizide yapılan değişiklik orijinal diziye yansır.
  - Dizinin tek bir elemanı (`myArray[3]`) ise sıradan bir `int`'tir ve değerle aktarılır. Fonksiyondaki değişiklik orijinale yansımaz.
- Bubble sort
  - Her turda yan yana iki elemana bakılır. Sıra bozuksa yer değiştirilir (swap).
  - Örnek (6, 8, 7, 5, 3): 1. turdan sonra en büyük eleman (8) en sona gider. 2. turdan sonra 7 sondan ikinci olur. Bir turda hiç yer değiştirme olmazsa dizi sıralıdır.
  - En fazla `n - 1` tur yapılır. Her turda `i` 0'dan `n - 2`'ye kadar gider ve `a[i]` ile `a[i+1]` karşılaştırılır.
  - İyileştirmeler (her turda son sıralanmış kısmı atlamak, değişiklik olmayan turda durmak) sonraki derslere bırakıldı.
- Sıralı arama (sequential search)
  - `while (i < size && a[i] != key) i++;` Döngü bittiğinde `i == size` ise eleman yoktur ve -1 döndürülür, değilse `i` döndürülür.
  - Bir fonksiyonda şarta bağlı birden fazla `return` olabilir.
- İkili arama (binary search)
  - Dizi sıralı olmalıdır. Sayısal Analiz'deki bisection yönteminin dizideki karşılığıdır.
  - `low = 0`, `high = size - 1`, `mid = (low + high) / 2`.
  - `while (low <= high && a[mid] != key)`: aranan değer ortadakinden büyükse `low = mid + 1`, küçükse `high = mid - 1`, sonra `mid` yeniden hesaplanır.
  - Döngüden sonra `high < low` ise eleman bulunamamıştır (-1), değilse `mid` döndürülür.
- Laboratuvar duyurusu
  - Laboratuvar pazartesi 14.00'te yapılacak, yaklaşık bir saat sürecek. Ardından ders devam edecek.
  - Kodlar online sistem yerine Google Form ile, ad, soyad, numara ve rar dosyası olarak yüklenecek. İki şubenin bütün kodları aynı havuzda kopya programından geçirilecek.
  - Beş asistan, her biri 35-40 öğrencinin kodunu kontrol edecek. Kimin kimi kontrol ettiği listede açıklanacak. İtirazlar ilgili asistana mail ile yapılacak. Çözüm laboratuvardan sonra ilan edilecek.
  - Fonksiyon kullanılmayacak, yalnızca `main` içinde yazılacak (diğer şube fonksiyonları henüz görmemiş olabilir).
  - Formu doldururken güncel bir mail adresi yazın. Yükleme onayı mail ile gelir. Yüklediğinizi ve kodun son hâlini gösteren ekran görüntüleri alın.

Hocanın Özellikle Vurguladığı Kısımlar

- Kod tekrarı en sık yapılan kod kusurlarındandır. Aynı kodu kopyalayıp yapıştırmak yerine fonksiyon yazılmalıdır.
- Argüman (çağrıda gönderilen) ile parametre (tanımda alınan) farklı kavramlardır.
- Bir fonksiyon tek bir iş yapmalı ve adıyla uyumlu olmalıdır.
- C'de bir fonksiyon tek bir değer döndürebilir ve dizi döndüremez.
- `modifyArray` / `modifyElement` örneği bu konunun en önemli örneğidir. PDF'ten tekrar tekrar yazılmalı, hata yapılmalı ve hatalar bulunmalıdır.

- Derste konuyla ilgili sorular sorulmalıdır. Diğer dillerden veya ileri konulardan gelen sorular, henüz bilmeyen arkadaşların kafasını karıştırmamak için ders sonuna bırakılmalıdır.
- Bundan sonra algoritmalar akış diyagramı yerine daha çok C kodu üzerinden ve fonksiyonlarla gösterilecektir.

Kısa Tekrar Notları

- `void f()`: değer döndürmez. `int f()`: `return` ile bir `int` döndürür.
- Prototip: `int square(int);` (main'den önce, sonunda noktalı virgül).
- Yerel değişkenler yalnızca kendi fonksiyonunda geçerlidir.
- Call by value: kopya gider, orijinal değişmez. Call by reference: adres gider, orijinal değişir.
- Dizi parametresi: `int a[]`, ayrıca `int size`. Dizi referansla, eleman değerle gider.
- Dizinin adı = 0. elemanın adresi.
- `int n[5] = {0};` hepsi 0. `char s[] = "first";` 6 karakter (`'\0'` dahil).
- Bubble sort: yan yana karşılaştır, gerekirse swap. Her turda en büyük sona gider.
- Binary search: sıralı dizi, `low`, `high`, `mid`. Bulunamazsa -1.

Detaylı Açıklamalar

`modifyArray` ve `modifyElement`

Slayttaki örnekte 5 elemanlı `a = {0, 1, 2, 3, 4}` dizisi kullanılır.

- `modifyArray(a, SIZE)` her elemanı 2 ile çarpar. Dönüş değeri yoktur (`void`), ama dizi referansla geldiği için `main`'deki dizi `{0, 2, 4, 6, 8}` olur.
- `modifyElement(a[3])` gelen değeri 2 ile çarpıp yazdırır. İçeride 12 görülür, ama `a[3]` değerle aktarıldığı için `main`'de hâlâ 6'dır.

Derste yazılan program

Derste Dev-C++'ta yazılan programın ekran görüntülerinden derlenmiş hâli (ders sonrasında `gcc -Wall -std=c89 -pedantic` ile derlenip çalıştırıldı):

```c
#include <stdio.h>
#define MAX 10
void bubble_sirala(int[], int);
void dizi_yazdir(int [], int);
void swap(int, int);
int sirali_ara(int [], int, int);
int binary_ara(int [], int, int);

int main(){
    int a[MAX]={20,54,7,8,9,11,13,14,18,19};
    int x=4, y=5;
    dizi_yazdir(a,MAX);
    bubble_sirala(a,MAX);
    dizi_yazdir(a,MAX);
    printf("%d - %d\n", x,y);
    swap(x,y);
    printf("%d - %d\n", x,y);
    printf("Sirali arama=%d\n",sirali_ara(a,MAX,14));
    printf("Sirali arama=%d\n",sirali_ara(a,MAX,88));
    printf("Binary arama=%d\n",binary_ara(a,MAX,14));
    printf("Binary arama=%d\n",binary_ara(a,MAX,88));
    return 0;
}
void swap(int a, int b){
    int t;
    t=a; a=b; b=t;
```

```
}
void bubble_sirala(int a[], int size){
    int tur, i, temp;
    for(tur=1;tur<size;tur++)
        for(i=0;i<size-1;i++)
            if(a[i]>a[i+1]){
                temp=a[i]; a[i]=a[i+1]; a[i+1]=temp;
            }
}
void dizi_yazdir(int a[], int size){
    int i;
    for(i=0;i<size;i++)
        printf("%2d-",a[i]);
    printf("\n");
}
int sirali_ara(int a[], int size, int key){
    int i=0;
    while(i<size && a[i]!=key)
        i++;
    printf("Counter for sequential=%d\n",i);
    if(i==size)
        return -1;
    else
        return i;
}
int binary_ara(int a[], int size, int key){
    int low=0, i=1;
    int high=size-1;
    int mid=(low+high)/2;
    while(low<=high && a[mid]!=key){
        i++;
        if(key>a[mid])
            low=mid+1;
        else
            high=mid-1;
        mid=(low+high)/2;
    }
    printf("Counter for binary=%d\n",i);
    if(high<low)
        return -1;
    else
        return mid;
}
```

Çıktı:

```
20-54- 7- 8- 9-11-13-14-18-19-
 7- 8- 9-11-13-14-18-19-20-54-
4 - 5
4 - 5
Counter for sequential=5
Sirali arama=5
Counter for sequential=10
Sirali arama=-1
Counter for binary=3
Binary arama=5
Counter for binary=5
Binary arama=-1
```

- Dizi `bubble_sirala`'da değişti ve bu değişiklik `main`'e yansıdı (dizi referansla gider).
- `swap(x, y)` sonrasında `x` ve `y` yine 4 ve 5'tir. `swap` içindeki `a` ve `b` yalnızca kopyalardır

(değerle çağırma). Bu sorun pointer'larla çözülecektir. Bubble sort, selection sort ve insertion sort'taki bütün yer değiştirmeler tek bir `swap` fonksiyonuyla yapılabilecektir.
- 14'ü sıralı arama 5 karşılaştırmada (indis 5), ikili arama 3 adımda buldu. 88'i sıralı arama 10 elemanın hepsine bakarak, ikili arama 5 adımda bulamadı.
- Derste `swap`'ın prototipi önce yazılmamıştı. Dev-C++ bu yüzden "conflicting types for 'swap' " ve "previous implicit declaration of 'swap' was here" uyarılarını verdi. Prototip eklenince uyarı kalkar.
- Interpolasyon araması (interpolation search) bir sonraki derse bırakıldı.

Ders sonrası notlar

- **Değerle çağırmanın amacı:** Derste iki yerde "fonksiyonun değişkeni değiştirmesini istiyorsanız call by value kullanırsınız" anlamına gelen ifadeler geçti. Doğrusu tersidir: fonksiyonun orijinal değişkeni değiştirmesini istemiyorsanız değerle çağırırsınız. Değiştirmesini istiyorsanız adresini gönderirsiniz (referansla çağırma). Hocanın verdiği `printf` ve `scanf` örnekleri de bunu gösterir.
- **Boş parantezli prototip:** Bir öğrenci prototipi `int square();` diye, parametre tipi yazmadan bildirmeyi sordu. Hoca bunun "hiç parametre almayan" bir fonksiyon anlamına geldiğini ve derleme hatası vereceğini söyledi. C'de (C23 öncesi) boş parantez "parametreleri belirtilmemiş" demektir. Derleyici çağrıdaki argümanları kontrol etmez ama hata da vermez. Ders sonrasında `int square();` bildirimi ve `square(5)` çağrısıyla yazılan program `gcc -std=c89` ile derlendi ve 25 yazdırdı. Hiç parametre almayan bir fonksiyon `int f(void);` diye bildirilir. Prototipte parametre tiplerini yazmak, derleyicinin hatalı çağrıları yakalayabilmesi için önemlidir.
- **Dizilerin aktarımı:** Dizilerin "referansla" gittiği söylenir. Teknik olarak C'de her şey değerle aktarılır: dizinin adı ilk elemanın adresine dönüşür ve bu adresin bir kopyası gönderilir. Fonksiyon bu adres üzerinden orijinal elemanlara eriştiği için değişiklikler orijinale yansır.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.